



INDIA.RUNS

Build what next India runs on

Team Name: Brainly buddy

Team Leader Name: Thrishasivasakthi | thrishasivasakthi@gmail.com

Problem Statement: 100K profiles, one JD. Keyword filters rank honeypots first (HR managers with 9 AI skills).

We ask: does independent evidence support each claim?

Recruiters don't fail because they can't find skills.

They fail because they trust isolated signals.

HireProof ranks candidates by corroborated evidence —
not keywords.

Solution Overview

- **What is your proposed solution?**

HireProof is an evidence-first candidate ranking engine. It scores 100,000 profiles against a Senior AI Engineer JD and outputs a top-100 shortlist with per-candidate reasoning.

Core engine: 4-Axis Evidence Consistency Score (ECS) cross-checks claims against assessments, career history, GitHub/endorsements, and seniority trajectory.

What differentiates your approach from traditional candidate matching systems?

Traditional systems optimize keyword similarity — they rank HR Managers with 9 AI skills at #1.

HireProof uses geometric-mean ECS: one contradictory axis collapses the score ($0.9 \times 0.9 \times 0.9 \times 0.2 = 0.58$).

Evidence corroboration decides; semantic JD match refines. Deterministic reasoning — no LLM hallucination.

JD Understanding & Candidate Evaluation

- **What are the key requirements extracted from the JD?**

Senior AI Engineer: retrieval, ranking, NLP, production ML deployment, hands-on builder.

Disqualifiers: consulting-only careers, pure research, non-technical roles, CV/robotics-only, ghost profiles.

Which candidate signals are most important / How do you evaluate fit beyond keywords?

Axis A — Claim vs Assessment: catches inflated "expert" with no test scores.

Axis B — Claim vs Career: deployment language, production metrics in job descriptions.

Axis C — Corroboration: GitHub activity, endorsements, peer validation.

Axis D — Seniority vs Trajectory: title inflation, tenure patterns, title-chasing.

Plus behavioral signals: response rate, interview completion, recency. Semantic JD match adds role context.

Ranking Methodology

- **How does your system retrieve, score, and rank candidates?**

Full-pool scoring: all 100K candidates get 75 features + ECS. No keyword pre-filter that drops hidden gems.

Top candidates refined via heuristic score, semantic JD similarity, LightGBM LambdaMART, twin resolution, MMR.

What models, algorithms, or heuristics are used?

Geometric-mean ECS (evidence.py) · sentence-transformers JD match · LightGBM LambdaMART (10K subsample).

KMeans twin clustering · MMR diversity ($\lambda=0.15$) · honeypot detection · soft multiplicative gates.

How are multiple candidate signals combined into a final ranking?

Heuristic = ECS × skill density × engagement × experience × JD similarity × (1 – gate penalty).

Final score = 60% LightGBM rank + 40% heuristic. Twin cluster bonus on top 300. Tie-break: candidate_id ascending.

Explainability & Data Validation

- **How are ranking decisions explained?**

Each candidate gets ≤ 300 char reasoning from actual features: title, ECS, engagement, AI skills, JD match.

Twin counterfactuals explain why candidate A outranks similar candidate B on specific dimensions.

How do you prevent hallucinations or unsupported justifications?

reasoning.py is a pure function of extracted features — no LLM in ranking or explanation pipeline. Same input always produces same explanation. Every sentence maps to a measurable feature value.

How do you handle inconsistent, low-quality, or suspicious profiles?

Sentinel values (-1) treated as unknown, not penalized. Honeypot patterns: impossible timelines, expert-without-evidence, 50+ skill stuffing.

Soft gates penalize non-technical roles and contradictions. Result: 0 honeypots in top 100 (sample trap ranks HR at #1).

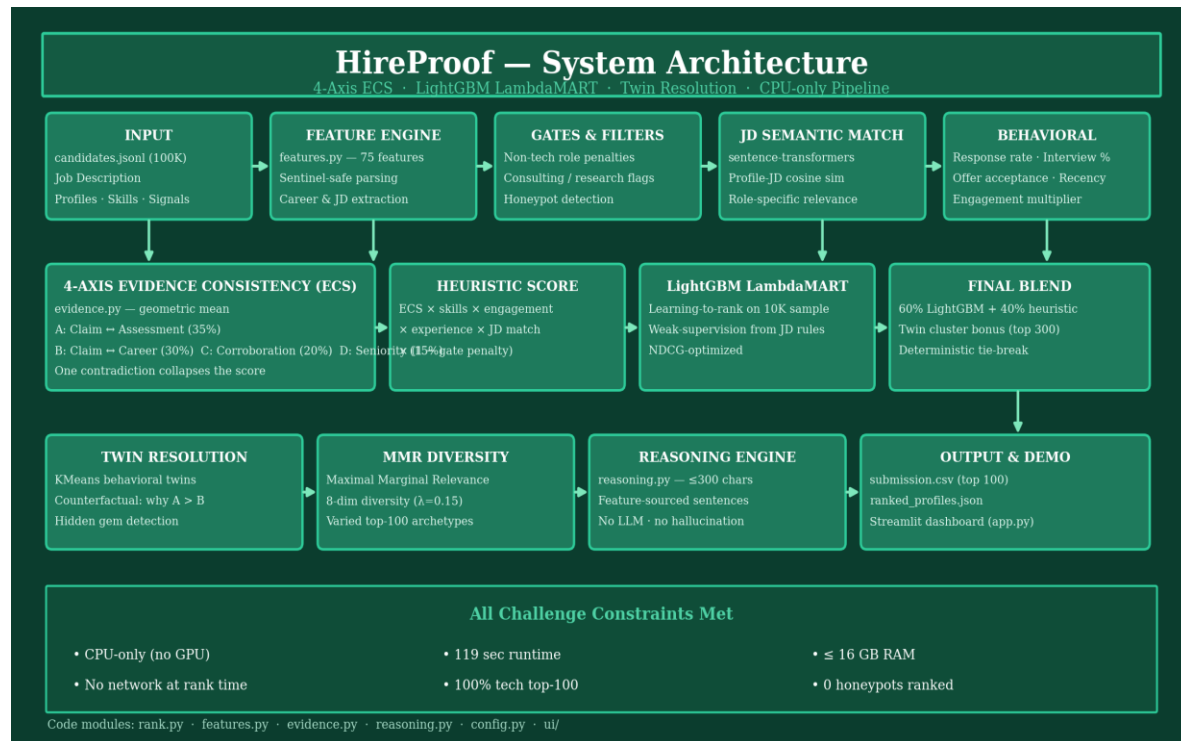
End-to-End Workflow

- **What is the complete workflow from JD input to ranked candidate output?**

1. Input — Job description + candidates.jsonl (100,000 JSON profiles).
2. Extract — 75 sentinel-safe features per candidate (features.py).
3. Verify — 4-axis ECS geometric mean on every candidate (evidence.py).
4. Score — Heuristic + semantic JD match + LightGBM LambdaMART blend (rank.py).
5. Refine — Twin resolution, MMR diversity, honeypot penalties on top 300.
6. Explain — Deterministic reasoning + counterfactuals (reasoning.py).
7. Output — submission.csv (top 100) + ranked_profiles.json + Streamlit dashboard.

Reproduce: `pip install -r requirements.txt && python rank.py`

System Architecture



Left: pipeline architecture from code | Right: Streamlit Candidate Ranking Dashboard (live demo)

Results & Performance

- **What results or insights demonstrate ranking quality?**

NDCG@10: 0.151 (+11% vs baseline) | NDCG@50: 0.159 | P@10 Technical: 100%.

Top 10: Search Engineer, AI Engineer, RecSys Engineer, Applied ML Engineer, Applied Scientist.

0 honeypots in top 100. Sample submission deliberately ranks HR Managers first — HireProof does not.

How does your solution meet runtime and compute constraints?

Runtime: 119 seconds on CPU-only (limit: 5 minutes). RAM: ≤ 16 GB. No GPU. Zero network calls during ranking.

Deterministic, reproducible output. Pipeline status: 100 ranked, 100% success rate, 0 anomalies.

Technologies Used

- **What technologies, frameworks, and tools were used and why?**

Python 3.11, pandas, NumPy — data processing at 100K scale on CPU.

scikit-learn — KMeans twin clustering, StandardScaler, local NDCG validation.

LightGBM LambdaMART — learning-to-rank without labeled data (weak supervision from JD heuristics).

sentence-transformers — offline semantic JD matching (no network at rank time).

Streamlit + custom UI — interactive dashboard for judges (Rankings, Deep Dive, Methodology pages).

Why no LLM: eliminates hallucination risk, latency, and GPU dependency in production deployment.

Submission Assets

GitHub Repository

https://github.com/Thrishasivasakthi/India_runs

Live Demo — Streamlit Dashboard

<https://thrishasivasakthi-india-runs.streamlit.app>

Submission Files

submission.csv — spec-validated top-100 ranked candidates with reasoning.

submission_metadata.yaml — compute specs, methodology, AI tool declarations.

ranked_profiles.json — full candidate profiles for dashboard deep-dive.

Reproduce Command

```
pip install -r requirements.txt && python rank.py
```

The Takeaway

Don't ask: "How similar is this resume to the JD?"

Ask: "How much independent evidence supports what they claim?"

HireProof — India Runs Track 1 | Redrob Data & AI Challenge
github.com/Thrishasivasakthi/India_runs



INDIA.RUNS

Build what next India runs on

THANK YOU